

Query 1:

--set enable_seqscan = off

Before index:

Data Output	Explain	Messages	Notifications
QUERY PLAN			
text			
1	Hash Join (cost=1.31..256.85 rows=298 width=175) (actual time=0.363..1.781 rows=403 loops=1)		
2	Hash Cond: (fd.fdgrp_cd = fg.fdgrp_cd)		
3	-> Seq Scan on food_des fd (cost=0.00..233.46 rows=7146 width=151) (actual time=0.006..0.638 rows=7146 loops=1)		
4	-> Hash (cost=1.30..1.30 rows=1 width=24) (actual time=0.010..0.012 rows=1 loops=1)		
5	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
6	-> Seq Scan on fd_group fg (cost=0.00..1.30 rows=1 width=24) (actual time=0.005..0.007 rows=1 loops=1)		
7	Filter: (fddrp_desc = 'Breakfast Cereals'::text)		
8	Rows Removed by Filter: 23		
9	Planning Time: 0.672 ms		
10	Execution Time: 1.828 ms		

Plan time = 0.672 ms

Execution time = 1.828 ms

Startup and Total Costs:

Sequential Scan:

Startup:0, total cost: 1.30 (number of rows remained the same 1)

Hash Join:

Startup: 1.30, total cost: 1.30 (number of rows remained the same 1)

Sequential Scan:

Startup: 0, total cost: 233.46 (number of rows remained the same 7146)

Hash Join:

Startup 1.31, total cost:256.85 (highest)

Number of Rows before index increased from 298 to 403.

Actual Time:

Sequential Join:

Startup:0.005, total time: 0.007

Hash Join:

Startup:0.010, total time: 0.012

Sequential Join:

Startup:0.006, total time: 0.638

Hash Join:

Startup:0.363, total time: 1.781 (slowest)

After index:

BTree index is created to optimize the equality searches and for better performance. It's created on "fddrp_desc".

CREATE INDEX idx_fdgrp_cd ON fd_group(fddrp_desc);

Plan time: 0.133 ms (decreased)

Execution time: 130.567 ms (increased)

	Data Output	Explain	Messages	Notifications
	QUERY PLAN ▲ text 🔒			
1	Hash Join (cost=10000000008.17..10000000263.70 rows=298 width=175) (actual time=128.367..129.083 rows=403 loops=1)			
2	Hash Cond: (fd.fmgrp_cd = fg.fmgrp_cd)			
3	-> Seq Scan on food_des fd (cost=10000000000.00..10000000233.46 rows=7146 width=151) (actual time=0.016..0.453 rows=7146 loops=1)			
4	-> Hash (cost=8.15..8.15 rows=1 width=24) (actual time=127.966..127.967 rows=1 loops=1)			
5	Buckets: 1024 Batches: 1 Memory Usage: 9kB			
6	-> Index Scan using idx_fd_fmgrp_cd on fd_group fg (cost=0.14..8.15 rows=1 width=24) (actual time=0.032..0.039 rows=1 loops=1)			
7	Index Cond: (fd.fmgrp_desc = 'Breakfast Cereals':text)			
8	Planning Time: 0.133 ms			
9	JIT:			
10	Functions: 12			
11	Options: Inlining true, Optimization true, Expressions true, Deforming true			
12	Timing: Generation 1.370 ms, Inlining 4.569 ms, Optimization 77.273 ms, Emission 45.804 ms, Total 129.017 ms			
13	Execution Time: 130.567 ms			

Plan time decreased however execution time increased meaning that PostgreSQL has chosen a less optimal plan to execute the query which can lead to a worse performance for the query.

Startup and Total Costs:

Index Scan:

Startup: 0.14, total cost: 8.15

(number of rows remained the same 1)

Hash:

Startup: 8.15, total cost: 8.15

(number of rows remained the same 1)

Sequential Scan:

Startup: 10000000000.00, total cost: 100000233.46

(number of rows remained the same 7146)

Hash Join:

Startup: 10000000008.17, total cost: 10000000263.70 (highest)

Number of rows increased from 298 to 403

Actual Time:

Index Scan:

Startup:0.032, total time: 0.039

Hash:

Startup:127.966, total time: 127.967

Sequential Join:

Startup:0.016, total time: 0.453

Hash Join:

Startup:128.367, total time: 129.083 (slowest)

Query 2:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Seq Scan on weight (cost=0.00..287.61 rows=6666 width=50) (actual time=0.015..1.497 rows=6666 loops=1)		
2	Filter: (seq = '1'::bpchar)		
3	Rows Removed by Filter: 6343		
4	Planning Time: 0.242 ms		
5	Execution Time: 1.701 ms		

Since that we only removed some rows by filtering and then another filter operation was used. Only sequential scan is used:

Plan Time: 0.242 ms

Execution Time: 1.701 ms

Startup cost: 0, total cost: 287.61 (highest)

Number of rows remained the same 6666

Startup time = 0.015, total time: 1.497 (slowest)

After Index:

BTree index is created for better performance and to optimize the where condition. It's created on "seq".

CREATE INDEX idx_seq ON weight(seq);

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Seq Scan on weight (cost=0.00..287.61 rows=6666 width=50) (actual time=0.014..1.537 rows=6666 loops=1)		
2	Filter: (seq = '1'::bpchar)		
3	Rows Removed by Filter: 6343		
4	Planning Time: 0.226 ms		
5	Execution Time: 1.786 ms		

Since that we only removed some rows by filtering and then another filter operation was used. Only sequential scan is used:

Plan Time: 0.226 ms (decreased)

Execution Time: 1.786 ms (increased)

Plan time decreased however execution time slightly increased as index is used better efficiency however there's a higher execution time as total time increased from 1.497 to 1.537 meaning that PostgreSQL might have chose a less optimal plan to execute the query leading to worse performance despite spending less time deciding how to execute it.

Startup cost: 0, total cost: 287.61 (remained the same) & (highest)

Number of rows remained the same 6666

Startup time = 0.014, total time: 1.537 (slower than before index)

Query 3:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Seq Scan on weight (cost=0.00..320.13 rows=144 width=50) (actual time=0.011..0.893 rows=60 loops=1)		
2	Filter: ((amount > '3'::double precision) AND (seq = '1'::bpchar))		
3	Rows Removed by Filter: 12949		
4	Planning Time: 0.125 ms		
5	Execution Time: 0.906 ms		

Since that we only removed some rows by filtering and then another filter operation was used. Only sequential scan is used:

Plan time: 0.125 ms

Execution time: 0.906 ms

Startup cost: 0, total cost: 320.13 (highest)

Number of rows decreased from 144 to 60

Startup time: 0.011, total time: 0.893 (slowest)

After Index:

BTree index is created for better performance and to optimize the where condition in “seq” and “amount”.

CREATE INDEX idx_seq_amount ON weight(seq, amount);

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Bitmap Heap Scan on weight (cost=5.76..139.14 rows=144 width=50) (actual time=0.030..0.063 rows=60 loops=1)		
2	Recheck Cond: ((seq = '1'::bpchar) AND (amount > '3'::double precision))		
3	Heap Blocks: exact=20		
4	-> Bitmap Index Scan on idx_seq_amount (cost=0.00..5.72 rows=144 width=0) (actual time=0.025..0.025 rows=60 loops=1)		
5	Index Cond: ((seq = '1'::bpchar) AND (amount > '3'::double precision))		
6	Planning Time: 0.154 ms		
7	Execution Time: 0.075 ms		

Plan time increased from 0.125 to 0.154. Execution time decreased from 0.906 to 0.075 indicating that the planner takes more time figuring out how to execute the query. Even though planning time increased, query has been optimized signaling that execution is much faster due to optimization and better performance.

Bitmap Index Scan:

Startup cost: 0, total: 5.72

Number of rows decreased from 144 to 60

Startup time: 0.025, total time: 0.025 (better than bitmap heap scan)

Bitmap Heap Scan:

Startup cost: 5.76, total cost: 139.14 (highest)

Number of rows decreased from 144 to 60

Startup time: 0.030, total time: 0.063 (slower than bitmap index scan) “slowest”

Query 4:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN			
text			
1	HashSetOp Except (cost=0.00..803.27 rows=6679 width=28) (actual time=6.994..6.996 rows=0 loops=1)		
2	-> Append (cost=0.00..752.89 rows=20155 width=28) (actual time=0.014..3.925 rows=20155 loops=1)		
3	-> Subquery Scan on "SELECT* 1" (cost=0.00..385.18 rows=13009 width=10) (actual time=0.013..1.877 rows=13009 loops=1)		
4	-> Seq Scan on weight (cost=0.00..255.09 rows=13009 width=6) (actual time=0.011..1.111 rows=13009 loops=1)		
5	-> Subquery Scan on "SELECT* 2" (cost=0.28..266.93 rows=7146 width=10) (actual time=0.080..1.207 rows=7146 loops=1)		
6	-> Index Only Scan using food_des_pkey on food_des (cost=0.28..195.47 rows=7146 width=6) (actual time=0.079..0.722 rows=7146 loops=1)		
7	Heap Fetches: 0		
8	Planning Time: 0.126 ms		
9	Execution Time: 7.099 ms		

Plan time: 0.126 ms

Execution time: 7.099 ms

- Index Join:

Startup time: 0.28, total cost: 195.47

Number of rows remained the same 7146

Startup time: 0.079, total time; 0.722

- Subquery Scan on select (2):

Startup cost: 0.28, total cost: 266.93

Number of rows remained the same 7146

Startup time: 0.080, total time: 1.207

- Sequential Scan on weight:

Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009.

Startup time: 0.011, total time: 1.111

- Subquery Scan on select (1):

Startup cost:0, total cost: 385.18

Number of rows remained the same 13009

Startup time:0.013 to 1.877

- (Append)

Startup cost:0, total cost: 752.89

Number of rows remained the same 20155

Startup time: 0.014, total time: 3.925

- HashSetOp Except join:
Startup cost: 0, total cost: 803.27 (highest)

Number of rows decreased from 6679 to 0.

Startup time: 6.994, total time: 6.996 (slowest)

After Index:

BTree indices are created to optimize the performance and the search. They're created on ndb_no.

```
CREATE INDEX idx_weight_ndb_no ON weight(ndb_no);
CREATE INDEX idx_food_des_ndb_no ON food_des(ndb_no);
```

Data Output	Explain	Messages	Notifications
QUERY PLAN text 1 HashSetOp Except (cost=0.00..803.27 rows=6679 width=28) (actual time=6.956..6.958 rows=0 loops=1) 2 -> Append (cost=0.00..752.89 rows=20155 width=28) (actual time=0.010..3.739 rows=20155 loops=1) 3 -> Subquery Scan on "SELECT* 1" (cost=0.00..385.18 rows=13009 width=10) (actual time=0.009..1.688 rows=13009 loops=1) 4 -> Seq Scan on weight (cost=0.00..255.09 rows=13009 width=6) (actual time=0.008..0.930 rows=13009 loops=1) 5 -> Subquery Scan on "SELECT* 2" (cost=0.28..266.93 rows=7146 width=10) (actual time=0.078..1.123 rows=7146 loops=1) 6 -> Index Only Scan using idx_food_des_ndb_no on food_des (cost=0.28..195.47 rows=7146 width=6) (actual time=0.076..0.701 rows=7146 loops=1) 7 Heap Fetches: 0 8 Planning Time: 0.325 ms 9 Execution Time: 7.014 ms			

Planning Time: 0.325 ms (increased)
Execution Time: 7.014 ms (decreased)

Plan time increased from 0.126 to 0.325. Execution time decreased from 7.099 to 7.014 indicating that the planner spends more time figuring out how to execute the query. Even though planning time increased, query has been optimized signaling that execution is much faster due to optimization and better performance.

- Index Scan join:
Startup cost: 0.28, total cost: 195.47

Number of rows remained the same 7146

Startup time: 0.076, total time: 0.701

- Subquery on select (2):
Startup cost: 0.28, total cost: 266.93

Number of rows remained the same 7146.

Startup time: 0.078, total time: 1.123

- Sequential Scan on weight:
Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009

Startup time: 0.008, total time: 0.930

- Subquery Scan on select (1):
Startup cost: 0, total cost: 385.18

Number of rows remained the same 13009

Startup time: 0.009, total time: 1.688

- (Append):

Startup cost: 0, total cost: 752.89

Number of rows remained the same 20155

Startup time: 0.010, total time: 3.739

- HashSetOp Except join:
Startup cost: 0, total cost: 803.27 (highest)

Number of rows decreased from 6679 to 0.

Startup time: 6.956, total time: 6.958 (slowest)

Query 5:
--set enable_seqscan = off

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN			
text			
1	HashSetOp Except (cost=0.00..3661.48 rows=6679 width=28) (actual time=31.713..32.014 rows=5077 loops=1)		
2	-> Append (cost=0.00..3394.35 rows=106854 width=28) (actual time=0.009..19.404 rows=106854 loops=1)		
3	-> Subquery Scan on "SELECT* 1" (cost=0.00..385.18 rows=13009 width=10) (actual time=0.008..1.832 rows=13009 loops=1)		
4	-> Seq Scan on weight (cost=0.00..255.09 rows=13009 width=6) (actual time=0.007..1.017 rows=13009 loops=1)		
5	-> Subquery Scan on "SELECT* 2" (cost=0.00..2474.90 rows=93845 width=10) (actual time=0.026..12.336 rows=93845 loops=1)		
6	-> Seq Scan on datsrcln (cost=0.00..1536.45 rows=93845 width=6) (actual time=0.025..6.600 rows=93845 loops=1)		
7	Planning Time: 0.059 ms		
8	Execution Time: 32.230 ms		

Plan Time: 0.059 ms

Execution Time: 32.230 ms

- Sequential Scan on datsrcln:
Startup cost: 0, total cost: 1536.45

Number of rows remained the same 93845

Startup time: 0.025, total time: 6.600

- Subquery on select (2):
Startup cost: 0, total cost: 2474.90

Number of rows remained the same 93845.

Startup time: 0.026, total time: 12.336.

- Sequential Scan on weight:
Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009

Startup time: 0.007, total time: 1.017

- Subquery Scan on select (1):
Startup cost: 0, total cost: 385.18

Number of rows remained the same 13009

Startup time: 0.008, total time: 1.832

- (Append):

Startup cost: 0, total cost: 3394.35

Number of rows remained the same 106854

Startup time: 0.009, total time: 19.404

- HashSetOp Except join:
Startup cost: 0, total cost: 3661.48 (highest)

Number of rows decreased from 6679 to 5077.

Startup time: 31.717, total time: 32.014 (slowest)

After Index:

BTree indices are created to optimize performance and search. They're created on `ndb_no`.

```
CREATE INDEX idx_weight_ndb_no ON weight(ndb_no);  
CREATE INDEX idx_food_des_ndb_no ON food_des(ndb_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	HashSetOp Except (cost=0.29..5473.46 rows=6679 width=28) (actual time=36.475..36.977 rows=5077 loops=1)		
2	-> Append (cost=0.29..5206.32 rows=106854 width=28) (actual time=0.077..24.302 rows=106854 loops=1)		
3	-> Subquery Scan on "SELECT* 1" (cost=0.29..453.51 rows=13009 width=10) (actual time=0.076..1.966 rows=13009 loops=1)		
4	-> Index Only Scan using idx_weight_ndb_no on weight (cost=0.29..323.42 rows=13009 width=6) (actual time=0.074..1.153 rows=13009 loops=1)		
5	Heap Fetches: 0		
6	-> Subquery Scan on "SELECT* 2" (cost=0.42..4218.54 rows=93845 width=10) (actual time=0.034..16.593 rows=93845 loops=1)		
7	-> Index Only Scan using idx_food_des_ndb_no on food_des (cost=0.42..3280.09 rows=93845 width=6) (actual time=0.033..10.437 rows=93845 loops=1)		
8	Heap Fetches: 0		
9	Planning Time: 0.131 ms		
10	Execution Time: 37.247 ms		

Plan Time: 0.131 ms (increased)

Execution Time: 37.247 ms (increased)

Both increased leading to inefficiencies or greater complexities in the query.

- Index scan:
Startup cost: 0.42, total cost: 4218.54

Number of rows remained the same 93845

Startup time: 0.033, total time: 10.437

- Subquery on select (2):
Startup cost: 0.42, total cost: 4218.54

Number of rows remained the same 93845.

Startup time: 0.034, total time: 16.593.

- Index scan on weight:
Startup cost: 0.29, total cost: 323.42

Number of rows remained the same 13009

Startup time: 0.074, total time: 1.153

- Subquery Scan on select (1):

Startup cost: 0.29, total cost: 453.51

Number of rows remained the same 13009

Startup time: 0.076, total time: 1.966

- (Append):

Startup cost: 0.29, total cost: 5206.32

Number of rows remained the same 106854

Startup time: 0.077, total time: 24.302

- HashSetOp Except join:

Startup cost: 0.29, total cost: 5473.46 (highest)

Number of rows decreased from 6679 to 5077.

Startup time: 36.475, total time: 36.977 (slowest)

Query 6:

Before Index:

	Data Output	Explain	Messages	Notifications
	QUERY PLAN text			
1	Hash Join (cost=5889.59..6229.80 rows=3633 width=20) (actual time=10.348..13.466 rows=4560 loops=1)			
2	Hash Cond: (w.ndb_no = nd.ndb_no)			
3	-> Seq Scan on weight w (cost=0.00..255.09 rows=13009 width=14) (actual time=0.006..0.841 rows=13009 loops=1)			
4	-> Hash (cost=5866.27..5866.27 rows=1866 width=18) (actual time=10.334..10.337 rows=2859 loops=1)			
5	Buckets: 4096 (originally 2048) Batches: 1 (originally 1) Memory Usage: 172kB			
6	-> Nested Loop (cost=0.42..5866.27 rows=1866 width=18) (actual time=0.019..9.750 rows=2859 loops=1)			
7	-> Seq Scan on nutr_def nutdef (cost=0.00..3.70 rows=1 width=16) (actual time=0.007..0.022 rows=1 loops=1)			
8	Filter: (tagname = 'CA':text)			
9	Rows Removed by Filter: 135			
10	-> Index Only Scan using nut_data_pkey on nut_data nd (cost=0.42..5843.34 rows=1923 width=10) (actual time=0.010..9.492 rows=2859 loops=1)			
11	Index Cond: (nutr_no = nutdef.nutr_no)			
12	Heap Fetches: 0			
13	Planning Time: 0.759 ms			
14	Execution Time: 13.632 ms			

Plan Time: 0.759 ms

Execution Time: 13.632 ms

- Index Scan:

Startup cost: 0.42, total cost: 5843.34

Number of rows increased from 1923 to 2859

Startup time: 0.010, total time: 9.492

- Sequential Scan:

Startup cost: 0, total cost: 3.70

Number of rows remained the same 1

Startup time: 0.007, total time: 0.022

- Nested Loop:

Startup cost: 0.42, total cost: 5866.27

Number of rows increased from 1866 to 2859

Startup time: 0.019, total time: 9.750

- Hash join:
Startup cost: 5866.27, total cost: 5866.37

Number of rows increased from 1866 to 2859

Startup time: 10.334, total time: 10.337

- Sequential Scan on weight w:
Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009

Startup time: 0.006, total time: 0.841

- Hash join:
Startup cost: 5889.59, total cost: 6229.80 (highest)

Number of rows increased from 3633 to 4560

Startup time: 10.348, total time: 13.466 (slowest)

After Index:

BTree indices are created to optimize performance and the join. They're created on tagname, nutr_no and ndb_no.

```
CREATE INDEX idx_tagname ON nutr_def(tagname);  
CREATE INDEX idx_nutr_no ON nut_data(nutr_no);  
CREATE INDEX idx_ndb_no ON weight(ndb_no);
```

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	Hash Join (cost=2794.94..3135.15 rows=3633 width=20) (actual time=3.673..6.220 rows=4560 loops=1)	
2	Hash Cond: (w.ndb_no = nd.ndb_no)	
3	-> Seq Scan on weight w (cost=0.00..255.09 rows=13009 width=14) (actual time=0.004..0.676 rows=13009 loops=1)	
4	-> Hash (cost=2771.62..2771.62 rows=1866 width=18) (actual time=3.659..3.661 rows=2859 loops=1)	
5	Buckets: 4096 (originally 2048) Batches: 1 (originally 1) Memory Usage: 172kB	
6	-> Nested Loop (cost=23.32..2771.62 rows=1866 width=18) (actual time=0.400..3.083 rows=2859 loops=1)	
7	-> Seq Scan on nutr_def nutdef (cost=0.00..3.70 rows=1 width=16) (actual time=0.006..0.022 rows=1 loops=1)	
8	Filter: (tagname = 'CA':text)	
9	Rows Removed by Filter: 135	
10	-> Bitmap Heap Scan on nut_data nd (cost=23.32..2748.69 rows=1923 width=10) (actual time=0.390..2.777 rows=2859 loops=1)	
11	Recheck Cond: (nutr_no = nutdef.nutr_no)	
12	Heap Blocks: exact=2091	
13	-> Bitmap Index Scan on idx_nut_data_nutr_no (cost=0.00..22.84 rows=1923 width=0) (actual time=0.202..0.203 rows=2859 loops=1)	
14	Index Cond: (nutr_no = nutdef.nutr_no)	
15	Planning Time: 0.815 ms	
16	Execution Time: 6.355 ms	

Plan Time: 0.815 ms (increased)

Execution Time: 6.355 ms (decreased)

Plan time increased from 0.759 to 0.815. Execution time decreased from 13.632 to 6.355 indicating planner spends more time figuring out how to execute the query. Even though planning time increased, query has been optimized signaling that execution is much faster due to optimization and better performance.

- Bitmap Index Scan:
Startup cost: 0, total cost: 22.84

Number of rows increased from 1923 to 2859

Startup time: 0.202, total time: 0.203

- Bitmap Heap Scan join:

Startup cost: 23.32, total cost: 2748.69

Number of rows increased from 1923 to 2859

Startup time: 0.390, total time: 2.777

- Sequential Scan:

Startup cost: 0, total cost: 3.70

Number of rows remained the same 1

Startup time: 0.006, total time: 0.022

- Nested Loop:

Startup cost: 23.32, total cost: 2771.62

Number of rows increased from 1866 to 2859

Startup time: 0.400, total time: 3.083

- Hash join:

Startup cost: 2771.62, total cost: 2771.62

Number of rows increased from 1866 to 2859

Startup time: 3.659, total time: 3.661

- Sequential Scan on weight w:

Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009

Startup time: 0.004, total time: 0.676

- Hash join:

Startup cost: 2794.94, total cost: 3135.15 (highest)

Number of rows increased from 3633 to 4560

Startup time: 3.673, total time: 6.220 (slowest)

Query 7:

--set enable_seqscan = off

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Hash Join (cost=6216.71..6799.84 rows=18169 width=20) (actual time=41.665..46.177 rows=28060 loops=1)		
2	Hash Cond: (w.ndb_no = nd.ndb_no)		
3	→ Seq Scan on weight w (cost=0.00..255.09 rows=13009 width=14) (actual time=0.003..0.622 rows=13009 loops=1)		
4	→ Hash (cost=6100.06..6100.06 rows=9332 width=18) (actual time=41.648..41.651 rows=14486 loops=1)		
5	Buckets: 16384 Batches: 1 Memory Usage: 851kB		
6	→ Hash Join (cost=3.76..6100.06 rows=9332 width=18) (actual time=0.019..38.971 rows=14486 loops=1)		
7	Hash Cond: (nd.nutr_no = nutdef.nutr_no)		
8	→ Seq Scan on nut_data nd (cost=0.00..5409.25 rows=253825 width=10) (actual time=0.002..14.564 rows=253825 loops=1)		
9	→ Hash (cost=3.70..3.70 rows=5 width=16) (actual time=0.014..0.015 rows=5 loops=1)		
10	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
11	→ Seq Scan on nutr_def nutdef (cost=0.00..3.70 rows=5 width=16) (actual time=0.006..0.012 rows=5 loops=1)		
12	Filter: (tagname ~ 'CA':text)		
13	Rows Removed by Filter: 131		
14	Planning Time: 0.405 ms		
15	Execution Time: 46.839 ms		
		Successfully run. Total query runtime: 90 msec. 15 rows affected.	

Plan time: 0.405 ms
Execution Time: 46.839 ms

● Sequential Scan:
Startup cost: 0, total cost: 3.70

Number of rows remained the same 5

Startup time: 0.006, total time: 0.012

● Hash:
Startup cost: 3.70, total cost: 3.70

Number of rows remained the same 5

Startup time: 0.014, total time: 0.015

● Sequential Scan:
Startup cost: 0, total cost: 5409.25

Number of rows remained the same 253825

Startup time: 0.002, total time: 14.564

● Hash Join:
Startup cost: 3.76, total cost: 6100.06

Number of rows increased from 9332 to 14486

Startup time: 0.019, total time: 38.971

● Hash:
Startup cost: 6100.06, total time: 6100.06

Number of rows increased from 9332 to 14486

Startup time: 41.648, total time: 41.651

● Sequential Scan on weight w:
Startup cost: 0, total cost: 255.09

Number of rows remained the same 13009

Startup time: 0.003, total time: 0.622

● Hash Join:
Startup cost: 6216.71, total cost: 6799.84 (highest)

Number of rows increased from 18169 to 28060.

Startup time: 41.665, total time: 46.177 (slowest)

After Index:

BTree indices are create to optimize join, performance and search. They're created on tagname, nutr_no and ndb_no.

```
CREATE INDEX idx_tagname ON nutr_def (tagname);
CREATE INDEX idx_nutr_no ON nut_data (nutr_no);
CREATE INDEX idx_ndb_no ON weight (ndb_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Hash Join (cost=637.50..8959.27 rows=18169 width=20) (actual time=4.098..20.225 rows=28060 loops=1)		
2	Hash Cond: (nd.ndb_no = w.ndb_no)		
3	-> Nested Loop (cost=23.47..7988.58 rows=9332 width=18) (actual time=0.413..11.568 rows=14486 loops=1)		
4	-> Index Scan using nutr_def_pkey on nutr_def nutdef (cost=0.14..15.52 rows=5 width=16) (actual time=0.006..0.049 rows=5 loops=1)		
5	Filter: (tagname ~~ 'CA%';text)		
6	Rows Removed by Filter: 131		
7	-> Bitmap Heap Scan on nut_data nd (cost=23.32..1575.38 rows=1923 width=10) (actual time=0.359..2.039 rows=2897 loops=5)		
8	Recheck Cond: (nutr_no = nutdef.nutr_no)		
9	Heap Blocks: exact=8953		
10	-> Bitmap Index Scan on idx_nutr_no (cost=0.00..22.84 rows=1923 width=0) (actual time=0.207..0.207 rows=2897 loops=5)		
11	Index Cond: (nutr_no = nutdef.nutr_no)		
12	-> Hash (cost=451.42..451.42 rows=13009 width=14) (actual time=3.665..3.666 rows=13009 loops=1)		
13	Buckets: 16384 Batches: 1 Memory Usage: 713kB		
14	-> Index Scan using idx_ndb_no on weight w (cost=0.29..451.42 rows=13009 width=14) (actual time=0.009..2.127 rows=13009 loops=1)		
15	Planning Time: 0.636 ms		
16	Execution Time: 22.392 ms		

Plan time: 0.636 ms (decreased)

Execution Time: 22.392 ms (increased)

Plan time decreased from 0.405 to 0.636 however execution time slightly increased as index is used better efficiency however there's a higher execution time as total time increased from 46.839 to 22.392 due to higher costs.

- Index Scan:

Startup cost: 0.29, total cost: 451.42

Number of rows remained the same 13009

Startup time: 0.009, total time: 2.127

- Hash:

Startup cost: 451.42, total cost: 451.42

Number of rows remained the same 13009

Startup time: 3.665, total time: 3.666

- Bitmap Index Scan:

Startup cost: 0, total cost: 22.84

Number of rows increased from 1923 to 2897

Startup time: 0.207, total time: 0.207

- Bitmap Heap Scan:

Startup cost: 23.32, total cost: 1575.38

Number of rows increased from 1923 to 2897

Startup time: 0.359, total time: 2.039

- Index Scan:

Startup cost: 0.14, total time: 15.52

Number of rows remained the same 5

Startup time:0.006, total time: 0.049

- Nested Loop:
Startup cost: 23.47, total cost: 7988.58

Number of rows increased from 9332 to 14486

Startup time: 0.413, total time: 11.568

- Hash Join:
Startup cost: 637.50, total cost: 8959.27 (highest)

Number of rows increased from 18169 to 28060.

Startup time: 4.098, total time: 20.225 (slowest)

Query 8:

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Aggregate (cost=238.24..238.25 rows=1 width=8) (actual time=1.099..1.100 rows=1 loops=1)		
2	-> Seq Scan on food_des (cost=0.00..233.46 rows=1911 width=0) (actual time=0.023..1.022 rows=1911 loops=1)		
3	Filter: (fat_factor IS NULL)		
4	Rows Removed by Filter: 5235		
5	Planning Time: 0.155 ms		
6	Execution Time: 1.134 ms		

Planning Time: 0.155 ms

Execution Time: 1.134 ms

Remove rows and then we filter then:

- Sequential Scan:
Startup cost: 0, total cost: 233.46

Number of rows remained the same 1911

Startup time: 0.023, total time: 1.022

- Count:
Startup cost: 238.24, total cost: 238.25 (highest)

Number of rows remained the same 1.

Startup time: 1.099, total time: 1.100 (slowest)

After Index:

Partial index is created to optimize the where condition, aggregate condition and the “fat_factor = null” condition. So it’s created on fat_factor.

CREATE INDEX idx_fat_factor ON food_des (fat_factor) WHERE fat_factor IS NULL;

Data Output	Explain	Messages	Notifications
QUERY PLAN text 1 Aggregate (cost=65.72..65.73 rows=1 width=8) (actual time=0.219..0.219 rows=1 loops=1) 2 -> Index Only Scan using idx_fat_factor on food_des (cost=0.28..60.94 rows=1911 width=0) (actual time=0.028..0.160 rows=1911 loops=1) 3 Heap Fetches: 0 4 Planning Time: 0.168 ms 5 Execution Time: 0.235 ms			

Plan time increased from 0.155 to 0.168. Execution time decreased from 1.134 to 0.235 indicating planner spends more time figuring out how to execute the query. Even though planning time increased, query has been optimized signaling that execution is much faster due to optimization and better performance.

Plan Time: 0.168 ms (increased)
Execution Time: 0.235 ms (decreased)

- Index Scan:
Startup cost: 0.28, total cost: 60.94

Number of rows remained the same 1911

Startup time: 0.028, total time: 0.160

- Count:
Startup cost: 65.72, total cost: 65.73 (highest)

Number of rows remained the same 1.

Startup time: 0.219, total time: 0.219 (slowest)

Query 9:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text 1 Seq Scan on food_des (cost=0.00..233.46 rows=1911 width=151) (actual time=0.025..1.064 rows=1911 loops=1) 2 Filter: (fat_factor IS NULL) 3 Rows Removed by Filter: 5235 4 Planning Time: 0.107 ms 5 Execution Time: 1.154 ms			

Plan Time: 0.107 ms
Execution Time: 1.154 ms

Remove rows by filtering then we filter as fat_factor is null then:

Sequential Scan:
Startup cost: 0, total time: 233.46 (highest)

Number of rows remained the same 1911

Startup time: 0.025, total time: 1.064 (slowest)

After Index:

Partial index is created to optimize the where condition and search. It's created on fat_factor.

CREATE INDEX idx_fat_factor ON food_des (fat_factor) WHERE fat_factor IS NULL;

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Bitmap Heap Scan on food_des (cost=42.31..223.42 rows=1911 width=151) (actual time=0.062..0.257 rows=1911 loops=1)		
2	Recheck Cond: (fat_factor IS NULL)		
3	Heap Blocks: exact=102		
4	-> Bitmap Index Scan on idx_fat_factor (cost=0.00..41.83 rows=1911 width=0) (actual time=0.052..0.052 rows=1911 loops=1)		
5	Planning Time: 0.119 ms		
6	Execution Time: 0.317 ms		

Plan Time: 0.119 ms (increased)

Execution Time: 0.317 ms (decreased)

Plan time increased from 0.107 to 0.119 however execution time slightly increased as planner spends more time figuring out how to execute the query however there's a lower execution time as total time increased from 1.154 to 0.317 due to optimization and better performance

Bitmap Index Scan:

Startup cost: 0, total cost: 41.83

Number of rows remained the same 1911

Startup time: 0.052, total time: 0.052

Bitmap Heap Scan:

Startup cost: 42.32, total time: 223.42 (highest)

Number of rows remained the same 1911

Startup time: 0.062, total time: 0.257 (slowest)

Query 10:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Aggregate (cost=238.24..238.25 rows=1 width=8) (actual time=1.065..1.066 rows=1 loops=1)		
2	-> Seq Scan on food_des (cost=0.00..233.46 rows=1911 width=8) (actual time=0.017..0.955 rows=1911 loops=1)		
3	Filter: (fat_factor IS NULL)		
4	Rows Removed by Filter: 5235		
5	Planning Time: 0.168 ms		
6	Execution Time: 1.087 ms		

Plan Time: 0.168 ms

Execution Time: 1.087 ms

Sequential Scan:

Startup cost: 0, total cost: 233.46

Number of rows remained the same 1911

Startup time: 0.017, total time: 0.955

Minimum:

Startup cost: 238.24, total cost: 238.25 (highest)

Number of rows remained the same 1

Startup time: 1.065, total time: 1.066 (slowest)

After Index:

Composite partial index is used to optimize minimum function, n_factor and fat_factor as well as the where condition.

```
CREATE INDEX idx_fat_factor ON food_des (fat_factor, n_factor) WHERE fat_factor IS NULL;
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Aggregate (cost=65.72..65.73 rows=1 width=8) (actual time=0.187..0.188 rows=1 loops=1)		
2	-> Index Only Scan using idx_fat_factor on food_des (cost=0.28..60.94 rows=1911 width=8) (actual time=0.021..0.134 rows=1911 loops=1)		
3	Heap Fetches: 0		
4	Planning Time: 0.320 ms		
5	Execution Time: 0.204 ms		

Plan Time: 0.320 ms (increased)

Execution Time: 0.204 ms (decreased)

Plan time increased from 0.090 to 0.320 however execution time slightly increased as planner spends more time figuring out how to execute the query however there's a lower execution time as total time increased from 1.042 to 0.207 due to optimization and better performance

Index Scan:

Startup cost: 0.28, total cost: 60.94

Number of rows remained the same 1911

Startup time: 0.021, total time: 0.134

Minimum:

Startup cost: 65.72, total cost: 65.73 (highest)

Number of rows remained the same 1

Startup time: 0.187, total time: 0.188 (slowest)

Query 11:

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Aggregate (cost=238.24..238.25 rows=1 width=8) (actual time=0.949..0.950 rows=1 loops=1)		
2	-> Seq Scan on food_des (cost=0.00..233.46 rows=1911 width=8) (actual time=0.024..0.867 rows=1911 loops=1)		
3	Filter: (fat_factor IS NULL)		
4	Rows Removed by Filter: 5235		
5	Planning Time: 0.104 ms		
6	Execution Time: 0.973 ms		

Plan Time: 0.104 ms
Execution Time: 0.973

Sequential Scan:
Startup cost: 0, total time: 233.46

Number of rows remained the same 1911

Startup time: 0.024, total time: 0.867

Sum:
Startup cost: 238.24, total cost: 238.25 (highest)

Number of rows remained the same 1

Startup time: 0.949, total time: 0.950 (slowest)

After Index:

Composite partial index is created to optimize both n_factor, fat_factor, where condition and the “sum function”.

CREATE INDEX idx_fat_factor ON food_des (fat_factor, n_factor) WHERE fat_factor IS NULL;

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Aggregate (cost=65.72..65.73 rows=1 width=8) (actual time=0.268..0.269 rows=1 loops=1)		
2	-> Index Only Scan using idx_fat_factor on food_des (cost=0.28..60.94 rows=1911 width=8) (actual time=0.034..0.193 rows=1911 loops=1)		
3	Heap Fetches: 0		
4	Planning Time: 0.235 ms		
5	Execution Time: 0.298 ms		

Plan Time: 0.235 ms (increased)
Execution Time: 0.298 ms (decreased)

Plan time increased from 0.104 to 0.235 however execution time slightly increased as planner spends more time figuring out how to execute the query however there's a lower execution time as total time increased from 0.973 to 0.298 due to optimization and better performance

Index Scan:
Startup cost: 0.28, total time: 60.94

Number of rows remained the same 1911

Startup time: 0.034, total time: 0.193

Sum:
Startup cost: 65.72, total cost: 65.73 (highest)

Number of rows remained the same 1

Startup time: 0.268, total time: 0.269 (slowest)

Query 12:

--set enable_seqscan = off

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN			
text			
1	Aggregate (cost=246.55..246.56 rows=1 width=8) (actual time=1.468..1.469 rows=1 loops=1)		
2	-> Seq Scan on food_des (cost=0.00..233.46 rows=5235 width=8) (actual time=0.007..1.153 rows=5235 loops=1)		
3	Filter: (fat_factor IS NOT NULL)		
4	Rows Removed by Filter: 1911		
5	Planning Time: 0.078 ms		
6	Execution Time: 1.491 ms		

Plan Time: 0.078 ms

Execution Time: 1.491 ms

Sequential Scan:

Startup cost: 0, total time: 233.46

Number of rows remained the same 5235

Startup time: 0.007, total time: 1.153

Sum (Aggregate):

Startup cost: 246.55, total cost: 246.56 (highest)

Number of rows remained the same 1

Startup time: 1.468, total time: 1.469 (slowest)

After Index:

Composite partial index is created to optimize both n_factor, fat_factor, where condition and the “sum function”. it is created on fat_factor and n_factor.

CREATE INDEX idx_fat_factor ON food_des (fat_factor, n_factor) WHERE fat_factor IS NOT NULL;

Data Output	Explain	Messages	Notifications
QUERY PLAN			
text			
1	Aggregate (cost=183.90..183.91 rows=1 width=8) (actual time=0.929..0.930 rows=1 loops=1)		
2	-> Index Only Scan using idx_fat_factor on food_des (cost=0.28..170.81 rows=5235 width=8) (actual time=0.027..0.644 rows=5235 loops=1)		
3	Heap Fetches: 0		
4	Planning Time: 0.227 ms		
5	Execution Time: 0.955 ms		

Plan Time: 0.227 ms (deccreased)

Execution Time: 0.955 ms (decreased)

Plan time increased from 0.078 to 0.227 however execution time slightly increased as planner spends more time figuring out how to execute the query however there's a lower execution time as total time increased from 1.491 to 0.955 due to optimization and better performance.

Index Scan:

Startup cost: 0.28, total time: 170.81

Number of rows remained the same 5235

Startup time: 0.027, total time: 0.644

Sum (Aggregate):

Startup cost: 183.90, total cost: 183.91 (highest)

Number of rows remained the same 1

Startup time: 0.929, total time: 0.930 (slowest)

Query 13:

Before Index:

Data Output Explain Messages Notifications

QUERY PLAN		
text		
1	Hash Join (cost=17.65..1803.98 rows=256 width=5) (actual time=0.131..9.073 rows=27 loops=1)	
2	Hash Cond: (dsl.nutr_no = nutdef.nutr_no)	
3	-> Hash Join (cost=12.59..1798.23 rows=256 width=4) (actual time=0.056..8.990 rows=27 loops=1)	
4	Hash Cond: (dsl.datasrc_id = ds.datasrc_id)	
5	-> Seq Scan on datsrcIn dsl (cost=0.00..1536.45 rows=93845 width=11) (actual time=0.004..3.469 rows=93845 loops=1)	
6	-> Hash (cost=12.57..12.57 rows=1 width=7) (actual time=0.028..0.030 rows=1 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	-> Seq Scan on data_src ds (cost=0.00..12.57 rows=1 width=7) (actual time=0.021..0.027 rows=1 loops=1)	
9	Filter: (voL_city = 'Cincinnati':text)	
10	Rows Removed by Filter: 365	
11	-> Hash (cost=3.36..3.36 rows=136 width=9) (actual time=0.055..0.056 rows=136 loops=1)	
12	Buckets: 1024 Batches: 1 Memory Usage: 14kB	
13	-> Seq Scan on nutr_def nutdef (cost=0.00..3.36 rows=136 width=9) (actual time=0.010..0.021 rows=136 loops=1)	
14	Planning Time: 0.526 ms	
15	Execution Time: 9.102 ms	

Seq Scan:

Startup cost: 0, total cost: 3.36

Rows remained the same 136

Startup time: 0.010, total time: 0.021

Hash:

Startup cost: 3.36, total cost: 3.36

Rows remained the same 136

Startup time: 0.055, total time: 0.056

Seq Scan:

Startup cost: 0, total cost: 12.57

Rows remained the same 1

Startup time: 0.021, total time: 0.027

Seq Scan on datsrcIn:

Startup cost: 0, total cost: 1536.45

Rows remained the same 93845

Startup time: 0.004, total time: 3.469

Hash Join:

Startup cost: 12.59, total cost: 1798.23

Rows decreased from 256 to 27

Startup time: 0.056, total time: 8.990

Hash Join:

Startup cost: 17.65, total cost: 1803.98 (highest)

Rows decreased from 256 to 27

Startup time: 0.131, total time: 9.073 (slowest)

After Index:

BTree indices are suitable in order to optimize range conditions, join, equality conditions and sorting operations. They'll be created on vol_city and nutr_no

```
CREATE INDEX idx_datasrc_vol_city ON data_src (vol_city);
CREATE INDEX idx_datsrcln_nutr_no ON datsrcln (nutr_no);
CREATE INDEX idx_nutrdef_nutr_no ON nutr_def (nutr_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Hash Join (cost=13.24..1799.57 rows=256 width=5) (actual time=0.089..11.347 rows=27 loops=1)		
2	Hash Cond: (dsl.nutr_no = nutdef.nutr_no)		
3	-> Hash Join (cost=8.18..1793.82 rows=256 width=4) (actual time=0.045..11.294 rows=27 loops=1)		
4	Hash Cond: (dsl.datasrc_id = ds.datasrc_id)		
5	-> Seq Scan on datsrcln dsl (cost=0.00..1536.45 rows=93845 width=11) (actual time=0.003..4.250 rows=93845 loops=1)		
6	-> Hash (cost=8.17..8.17 rows=1 width=7) (actual time=0.016..0.017 rows=1 loops=1)		
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
8	-> Index Scan using idx_datasrc_vol_city on data_src ds (cost=0.15..8.17 rows=1 width=7) (actual time=0.015..0.015 rows=1 loops=1)		
9	Index Cond: (vol_city = 'Cincinnati':text)		
10	-> Hash (cost=3.36..3.36 rows=136 width=9) (actual time=0.040..0.041 rows=136 loops=1)		
11	Buckets: 1024 Batches: 1 Memory Usage: 14kB		
12	-> Seq Scan on nutr_def nutdef (cost=0.00..3.36 rows=136 width=9) (actual time=0.009..0.021 rows=136 loops=1)		
13	Planning Time: 0.689 ms		
14	Execution Time: 11.379 ms		

Query 14

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Seq Scan on weight (cost=0.00..287.61 rows=11624 width=50) (actual time=0.010..0.957 rows=11625 loops=1)		
2	Filter: (amount < '2':double precision)		
3	Rows Removed by Filter: 1384		
4	Planning Time: 0.255 ms		
5	Execution Time: 1.197 ms		

After Index:

Create a BTree index on amount to optimize where condition and range condition.

```
CREATE INDEX idx_weight_amount ON weight (amount);
```

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Seq Scan on weight (cost=0.00..287.61 rows=11624 width=50) (actual time=0.005..0.884 rows=11625 loops=1)		
2	Filter: (amount < '2'::double precision)		
3	Rows Removed by Filter: 1384		
4	Planning Time: 0.128 ms		
5	Execution Time: 1.091 ms		

Planning Time = 0.128 ms
Execution Time = 1.091 ms

Since rows were removed by filtering.
Seq scan on weight:
Startup cost: 0, total cost: 287.61 (highest)
Startup time: 0.005, total time: 0.884 (slowest)
Number of rows remained the same 11625.

Query 15

Before Index:

Data Output	Explain	Messages	Notifications
QUERY PLAN text			
1	Seq Scan on weight (cost=0.00..287.61 rows=1215 width=50) (actual time=0.007..0.612 rows=1215 loops=1)		
2	Filter: (amount > '2'::double precision)		
3	Rows Removed by Filter: 11794		
4	Planning Time: 0.103 ms		
5	Execution Time: 0.651 ms		

Plan Time = 0.103 ms
Execution Time = 0.651 ms

Since that 11794 rows were removed by filtering then we filtered due to where condition, Seq Scan on weight was only used.

Seq Scan on weight:

Startup cost: 0, total cost: 287.61
Startup time: 0.007, total time: 0.612
Number of rows remained the same 1215.

After Index:

Create a BTree index on amount to optimize where condition and range condition.

```
CREATE INDEX idx_weight_amount ON weight (amount);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Bitmap Heap Scan on weight (cost=25.70..165.89 rows=1215 width=50) (actual time=0.050..0.189 rows=1215 loops=1)		
2	Recheck Cond: (amount > '2'::double precision)		
3	Heap Blocks: exact=86		
4	-> Bitmap Index Scan on idx_weight_amount (cost=0.00..25.40 rows=1215 width=0) (actual time=0.041..0.041 rows=1215 loops=1)		
5	Index Cond: (amount > '2'::double precision)		
6	Planning Time: 0.221 ms		
7	Execution Time: 0.225 ms		

Planning Time = 0.221 ms (increased)
Execution Time = 0.225 ms (decreased)

Bitmap Index Scan:
Startup cost: 0, total cost: 25.40
Startup time: 0.041, total time: 0.041
Number of rows remained the same 1215.

Bitmap Heap Scan on weight:
Startup cost: 25.70, total cost: 165.89 (highest)
Startup time: 0.050, total time: 0.189 (slowest)
Number of rows remained the same 1215.

Plan time increased from 0.103 to 0.221 however execution time slightly increased as planner spends more time figuring out how to execute the query however there's a lower execution time as total time increased from 0.651 to 0.225 due to optimization and better performance

Query 16

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	HashAggregate (cost=296.73..307.56 rows=867 width=29) (actual time=0.753..0.768 rows=140 loops=1)		
2	Group Key: amount, msre_desc		
3	Batches: 1 Memory Usage: 73kB		
4	-> Seq Scan on weight (cost=0.00..287.61 rows=1215 width=21) (actual time=0.007..0.571 rows=1215 loops=1)		
5	Filter: (amount > '2'::double precision)		
6	Rows Removed by Filter: 11794		
7	Planning Time: 0.150 ms		
8	Execution Time: 0.803 ms		

Plan Time: 0.150 ms
Execution Time: 0.803 ms

11794 rows were removed due to filtering then we filtered due to where condition.

Seq Scan on weight:

Startup cost: 0, total cost: 287.61
Startup time: 0.007, total time: 0.571
Number of rows remained the same 1215.

HashAggregate "avg"

Startup cost: 296.73, total cost: 307.56 (highest)
Startup time: 0.753, total time: 0.768 (slowest)
Number of rows decreased from 867 to 140.

After Index:

Create a BTree index on amount and msre_desc to optimize where condition, range condition and the grouping condition.

CREATE INDEX idx_weight_amount_msre_desc ON weight (amount, msre_desc);

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	GroupAggregate (cost=0.29..69.50 rows=867 width=29) (actual time=0.055..0.349 rows=140 loops=1)		
2	Group Key: amount, msre_desc		
3	-> Index Only Scan using idx_weight_amount_msre_desc on weight (cost=0.29..49.55 rows=1215 width=21) (actual time=0.044..0.191 rows=1215 loops=1)		
4	Index Cond: (amount > '2'::double precision)		
5	Heap Fetches: 0		
6	Planning Time: 0.305 ms		
7	Execution Time: 0.397 ms		

Query 17

Before Index

	QUERY PLAN text		Please
1	Group (cost=11846.58..11964.21 rows=1599 width=58) (actual time=34.485..35.925 rows=1 loops=1)		
2	Group Key: fd.long_desc, nd.num_studies		
3	InitPlan 1 (returns \$1)		
4	-> Finalize Aggregate (cost=5737.47..5737.48 rows=1 width=4) (actual time=22.533..22.673 rows=1 loops=1)		
5	-> Gather (cost=5737.36..5737.47 rows=1 width=4) (actual time=22.528..22.667 rows=2 loops=1)		
6	Workers Planned: 1		
7	Workers Launched: 1		
8	-> Partial Aggregate (cost=4737.36..4737.37 rows=1 width=4) (actual time=19.796..19.798 rows=1 loops=2)		
9	-> Parallel Seq Scan on nut_data (cost=0.00..4364.09 rows=149309 width=4) (actual time=0.005..7.338 rows=126912 loops=2)		
10	-> Gather Merge (cost=6109.10..6222.02 rows=941 width=58) (actual time=34.483..35.780 rows=1 loops=1)		
11	Workers Planned: 1		
12	Params Evaluated: \$1		
13	Workers Launched: 1		
14	-> Group (cost=5109.09..5116.15 rows=941 width=58) (actual time=10.465..10.469 rows=0 loops=2)		
15	Group Key: fd.long_desc, nd.num_studies		
16	-> Sort (cost=5109.09..5111.44 rows=941 width=58) (actual time=10.462..10.465 rows=0 loops=2)		
17	Sort Key: fd.long_desc		
18	Sort Method: quicksort Memory: 25kB		
19	Worker 0: Sort Method: quicksort Memory: 25kB		
20	-> Hash Join (cost=322.79..5062.62 rows=941 width=58) (actual time=7.606..10.414 rows=0 loops=2)		
21	Hash Cond: (nd.ndb_no = fd.ndb_no)		
22	-> Parallel Seq Scan on nut_data nd (cost=0.00..4737.36 rows=941 width=10) (actual time=6.820..9.624 rows=0 loops=2)		
23	Filter: (num_studies = \$1)		
24	Rows Removed by Filter: 126912		
25	-> Hash (cost=233.46..233.46 rows=7146 width=60) (actual time=1.552..1.554 rows=7146 loops=1)		
25	-> Hash (cost=233.46..233.46 rows=7146 width=60) (actual time=1.263..1.263 rows=7146 loops=1)		
26	Buckets: 8192 Batches: 1 Memory Usage: 708kB		
27	-> Seq Scan on food_des fd (cost=0.00..233.46 rows=7146 width=60) (actual time=0.008..0.504 rows=7146 loops=1)		
28	Planning Time: 0.271 ms		
29	Execution Time: 32.752 ms		

After Index

Create BTree indices on ndb_no and num_studies to optimize join and having conditions

CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);

CREATE INDEX idx_nut_data_ndb_no ON nut_data (ndb_no);

CREATE INDEX idx_nut_data_num_studies ON nut_data (num_studies);

Data Output Explain Messages Notifications

	QUERY PLAN text	
1	HashAggregate (cost=2902.86..2918.85 rows=1599 width=58) (actual time=2.462..2.479 rows=1 loops=1)	
2	Group Key: fd.long_desc, nd.num_studies	
3	Batches: 1 Memory Usage: 73kB	
4	InitPlan 2 (returns \$1)	
5	-> Result (cost=0.44..0.45 rows=1 width=4) (actual time=0.020..0.022 rows=1 loops=1)	
6	InitPlan 1 (returns \$0)	
7	-> Limit (cost=0.42..0.44 rows=1 width=4) (actual time=0.019..0.020 rows=1 loops=1)	
8	-> Index Only Scan Backward using idx_nut_data_num_studies on nut_data (cost=0.42..572.00 rows=27176 width=4) (actual time=0.018..0.019 rows=1 loops=1)	
9	Index Cond: (num_studies IS NOT NULL)	
10	Heap Fetches: 0	
11	-> Hash Join (cost=343.60..2894.41 rows=1599 width=58) (actual time=2.443..2.449 rows=1 loops=1)	
12	Hash Cond: (nd.ndb_no = fd.ndb_no)	
13	-> Bitmap Heap Scan on nut_data nd (cost=20.81..2567.43 rows=1599 width=10) (actual time=0.029..0.034 rows=1 loops=1)	
14	Recheck Cond: (num_studies = \$1)	
15	Heap Blocks: exact=1	
16	-> Bitmap Index Scan on idx_nut_data_num_studies (cost=0.00..20.41 rows=1599 width=0) (actual time=0.025..0.026 rows=1 loops=1)	
17	Index Cond: (num_studies = \$1)	
18	-> Hash (cost=233.46..233.46 rows=7146 width=60) (actual time=2.395..2.396 rows=7146 loops=1)	
19	Buckets: 8192 Batches: 1 Memory Usage: 708kB	
20	-> Seq Scan on food_des fd (cost=0.00..233.46 rows=7146 width=60) (actual time=0.007..1.064 rows=7146 loops=1)	
21	Planning Time: 0.527 ms	
22	Execution Time: 2.560 ms	

Query 18

Before Index:

Data Output Explain Messages Notifications

	QUERY PLAN text	
1	Hash Join (cost=1.54..257.07 rows=7146 width=175) (actual time=0.016..1.708 rows=7146 loops=1)	
2	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)	
3	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=151) (actual time=0.003..0.333 rows=7146 loops=1)	
4	-> Hash (cost=1.24..1.24 rows=24 width=24) (actual time=0.006..0.007 rows=24 loops=1)	
5	Buckets: 1024 Batches: 1 Memory Usage: 10kB	
6	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=24) (actual time=0.001..0.002 rows=24 loops=1)	
7	Planning Time: 0.228 ms	
8	Execution Time: 1.841 ms	

After Index:

Create BTree indices to optimize join condition & equal condition.

CREATE INDEX idx_fd_group_fdgrp_cd ON fd_group (fdgrp_cd);

CREATE INDEX idx_food_des_fdgrp_cd ON food_des (fdgrp_cd);

Data Output Explain Messages Notifications

	QUERY PLAN text	
1	Hash Join (cost=1.54..257.07 rows=7146 width=175) (actual time=0.016..1.709 rows=7146 loops=1)	
2	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)	
3	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=151) (actual time=0.004..0.332 rows=7146 loops=1)	
4	-> Hash (cost=1.24..1.24 rows=24 width=24) (actual time=0.007..0.008 rows=24 loops=1)	
5	Buckets: 1024 Batches: 1 Memory Usage: 10kB	
6	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=24) (actual time=0.001..0.003 rows=24 loops=1)	
7	Planning Time: 0.254 ms	
8	Execution Time: 1.873 ms	

Query 19

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Hash Join (cost=324.33..7184.21 rows=253825 width=267) (actual time=1.435..130.721 rows=253825 loops=1)		
2	Hash Cond: (fod.fmgrp_cd = fd.fmgrp_cd)		
3	-> Hash Join (cost=322.79..6398.73 rows=253825 width=243) (actual time=1.390..87.089 rows=253825 loops=1)		
4	Hash Cond: (nd.ndb_no = fod.ndb_no)		
5	-> Seq Scan on nut_data nd (cost=0.00..5409.25 rows=253825 width=92) (actual time=0.006..12.531 rows=253825 loops=1)		
6	-> Hash (cost=233.46..233.46 rows=7146 width=151) (actual time=1.358..1.359 rows=7146 loops=1)		
7	Buckets: 8192 Batches: 1 Memory Usage: 1364kB		
8	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=151) (actual time=0.003..0.328 rows=7146 loops=1)		
9	-> Hash (cost=1.24..1.24 rows=24 width=24) (actual time=0.014..0.015 rows=24 loops=1)		
10	Buckets: 1024 Batches: 1 Memory Usage: 10kB		
11	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=24) (actual time=0.005..0.007 rows=24 loops=1)		
12	Planning Time: 0.801 ms		
13	Execution Time: 135.757 ms		

After Index:

Create BTree indices to optimize the two join conditions.

```
CREATE INDEX idx_fd_group_fmgrp_cd ON fd_group (fmgrp_cd);
CREATE INDEX idx_food_des_fmgrp_cd ON food_des (fmgrp_cd);
CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);
CREATE INDEX idx_nut_data_ndb_no ON nut_data (ndb_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Hash Join (cost=324.33..7184.21 rows=253825 width=267) (actual time=2.012..171.077 rows=253825 loops=1)		
2	Hash Cond: (fod.fmgrp_cd = fd.fmgrp_cd)		
3	-> Hash Join (cost=322.79..6398.73 rows=253825 width=243) (actual time=1.961..112.242 rows=253825 loops=1)		
4	Hash Cond: (nd.ndb_no = fod.ndb_no)		
5	-> Seq Scan on nut_data nd (cost=0.00..5409.25 rows=253825 width=92) (actual time=0.009..15.846 rows=253825 loops=1)		
6	-> Hash (cost=233.46..233.46 rows=7146 width=151) (actual time=1.931..1.933 rows=7146 loops=1)		
7	Buckets: 8192 Batches: 1 Memory Usage: 1364kB		
8	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=151) (actual time=0.003..0.782 rows=7146 loops=1)		
9	-> Hash (cost=1.24..1.24 rows=24 width=24) (actual time=0.032..0.032 rows=24 loops=1)		
10	Buckets: 1024 Batches: 1 Memory Usage: 10kB		
11	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=24) (actual time=0.008..0.010 rows=24 loops=1)		
12	Planning Time: 0.714 ms		
13	Execution Time: 177.612 ms		

Query 20

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	HashSetOp Intersect (cost=0.29..267.70 rows=1 width=28) (actual time=0.844..0.846 rows=0 loops=1)		
2	-> Append (cost=0.29..266.95 rows=299 width=28) (actual time=0.659..0.840 rows=118 loops=1)		
3	-> Subquery Scan on "SELECT* 1" (cost=0.29..5.63 rows=1 width=10) (actual time=0.004..0.004 rows=0 loops=1)		
4	-> Index Scan using idx_weight_amount on weight (cost=0.29..5.62 rows=1 width=6) (actual time=0.004..0.004 rows=0 loops=1)		
5	Index Cond: (amount > '50':double precision)		
6	-> Subquery Scan on "SELECT* 2" (cost=1.31..259.83 rows=298 width=10) (actual time=0.654..0.830 rows=118 loops=1)		
7	-> Hash Join (cost=1.31..256.85 rows=298 width=6) (actual time=0.653..0.823 rows=118 loops=1)		
8	Hash Cond: (fod.fmgrp_cd = fd.fmgrp_cd)		
9	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=11) (actual time=0.005..0.335 rows=7146 loops=1)		
10	-> Hash (cost=1.30..1.30 rows=1 width=5) (actual time=0.007..0.008 rows=1 loops=1)		
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
12	-> Seq Scan on fd_group fd (cost=0.00..1.30 rows=1 width=5) (actual time=0.005..0.005 rows=1 loops=1)		
13	Filter: (fdgrp_desc = 'Snacks':text)		
14	Rows Removed by Filter: 23		
15	Planning Time: 0.197 ms		
16	Execution Time: 0.878 ms		

After Index:

Create a BTree index to optimize the where or filter condition.

```
CREATE INDEX idx_weight_amount ON weight (amount);
CREATE INDEX idx_group_fmgrp_cd ON fd_group (fmgrp_cd);
CREATE INDEX idx_food_des_fmgrp_cd ON food_des (fmgrp_cd);
CREATE INDEX idx_fd_group_fmgrp_desc ON fd_group (fmgrp_desc);
CREATE INDEX idx_weight_ndb_no ON weight (ndb_no);
CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	HashSetOp Intersect (cost=0.29..176.98 rows=1 width=28) (actual time=0.085..0.086 rows=0 loops=1)		
2	-> Append (cost=0.29..176.23 rows=299 width=28) (actual time=0.028..0.078 rows=118 loops=1)		
3	-> Subquery Scan on "SELECT* 1" (cost=0.29..5.63 rows=1 width=10) (actual time=0.004..0.004 rows=0 loops=1)		
4	-> Index Scan using idx_weight_amount on weight (cost=0.29..5.62 rows=1 width=6) (actual time=0.004..0.004 rows=0 loops=1)		
5	Index Cond: (amount > '50':double precision)		
6	-> Subquery Scan on "SELECT* 2" (cost=0.28..169.11 rows=298 width=10) (actual time=0.023..0.067 rows=118 loops=1)		
7	-> Nested Loop (cost=0.28..166.13 rows=298 width=6) (actual time=0.023..0.059 rows=118 loops=1)		
8	-> Seq Scan on fd_group fd (cost=0.00..1.30 rows=1 width=5) (actual time=0.008..0.008 rows=1 loops=1)		
9	Filter: (fdgrp_desc = 'Snacks':text)		
10	Rows Removed by Filter: 23		
11	-> Index Scan using idx_food_des_fmgrp_cd on food_des fod (cost=0.28..161.85 rows=298 width=11) (actual time=0.014..0.040 rows=118 loops=1)		
12	Index Cond: (fdgrp_cd = fd.fmgrp_cd)		
13	Planning Time: 0.503 ms		
14	Execution Time: 0.113 ms		

Query 21

Before Index:

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	Aggregate (cost=274.94..274.95 rows=1 width=32) (actual time=2.165..2.167 rows=1 loops=1)	
2	-> Hash Join (cost=1.54..257.07 rows=7146 width=6) (actual time=0.016..1.315 rows=7146 loops=1)	
3	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)	
4	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=11) (actual time=0.003..0.339 rows=7146 loops=1)	
5	-> Hash (cost=1.24..1.24 rows=24 width=5) (actual time=0.007..0.007 rows=24 loops=1)	
6	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
7	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=5) (actual time=0.002..0.004 rows=24 loops=1)	
8	Planning Time: 0.161 ms	
9	Execution Time: 2.191 ms	

After Index:

Create indices to optimize join conditions and aggregate function “max”.

```
CREATE INDEX idx_fd_group_fdgrp_cd ON fd_group (fdgrp_cd);
CREATE INDEX idx_food_des_fdgrp_cd ON food_des (fdgrp_cd);
CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);
```

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	Aggregate (cost=274.94..274.95 rows=1 width=32) (actual time=3.416..3.420 rows=1 loops=1)	
2	-> Hash Join (cost=1.54..257.07 rows=7146 width=6) (actual time=0.027..2.007 rows=7146 loops=1)	
3	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)	
4	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=11) (actual time=0.006..0.513 rows=7146 loops=1)	
5	-> Hash (cost=1.24..1.24 rows=24 width=5) (actual time=0.014..0.015 rows=24 loops=1)	
6	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
7	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=5) (actual time=0.005..0.008 rows=24 loops=1)	
8	Planning Time: 0.224 ms	
9	Execution Time: 3.456 ms	

Query 22

Before Index:

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	HashAggregate (cost=310.67..382.13 rows=7146 width=38) (actual time=4.399..6.223 rows=7146 loops=1)	
2	Group Key: fod.ndb_no	
3	Batches: 1 Memory Usage: 1425kB	
4	-> Hash Join (cost=1.54..257.07 rows=7146 width=22) (actual time=0.023..2.545 rows=7146 loops=1)	
5	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)	
6	-> Seq Scan on food_des fod (cost=0.00..233.46 rows=7146 width=27) (actual time=0.009..0.537 rows=7146 loops=1)	
7	-> Hash (cost=1.24..1.24 rows=24 width=5) (actual time=0.008..0.010 rows=24 loops=1)	
8	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
9	-> Seq Scan on fd_group fd (cost=0.00..1.24 rows=24 width=5) (actual time=0.002..0.004 rows=24 loops=1)	
10	Planning Time: 0.157 ms	
11	Execution Time: 6.620 ms	

After Index:

Create BTree indices to optimize join conditions and aggregate function “sum”.

```
CREATE INDEX idx_fd_group_fdgrp_cd ON fd_group (fdgrp_cd);
CREATE INDEX idx_food_des_fdgrp_cd ON food_des (fdgrp_cd);
CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);
```

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	HashAggregate (cost=448.94..520.40 rows=7146 width=38) (actual time=3.867..6.039 rows=7146 loops=1)		
2	Group Key: fod.ndb_no		
3	Batches: 1 Memory Usage: 1425kB		
4	-> Hash Join (cost=13.08..395.34 rows=7146 width=22) (actual time=0.021..2.470 rows=7146 loops=1)		
5	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)		
6	-> Index Scan using idx_food_des_ndb_no on food_des fod (cost=0.28..360.47 rows=7146 width=27) (actual time=0.006..1.013 rows=7146 loops=1)		
7	-> Hash (cost=12.50..12.50 rows=24 width=5) (actual time=0.009..0.012 rows=24 loops=1)		
8	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
9	-> Index Only Scan using idx_fd_group_fdgrp_cd on fd_group fd (cost=0.14..12.50 rows=24 width=5) (actual time=0.001..0.005 rows=24 loops=1)		
10	Heap Fetches: 24		
11	Planning Time: 0.210 ms		
12	Execution Time: 6.341 ms		

Query 23

```
-- set enable_seqscan = off
-- set enable_hashagg = off
```

Before Index:

Data Output	Explain	Messages	Notifications
	QUERY PLAN text		
1	Aggregate (cost=609.72..609.73 rows=1 width=16) (actual time=5.294..5.297 rows=1 loops=1)		
2	-> HashAggregate (cost=431.07..502.53 rows=7146 width=38) (actual time=4.077..5.044 rows=7146 loops=1)		
3	Group Key: fod.ndb_no		
4	Batches: 1 Memory Usage: 1169kB		
5	-> Hash Join (cost=13.08..395.34 rows=7146 width=22) (actual time=0.030..2.518 rows=7146 loops=1)		
6	Hash Cond: (fod.fdgrp_cd = fd.fdgrp_cd)		
7	-> Index Scan using food_des_pkey on food_des fod (cost=0.28..360.47 rows=7146 width=27) (actual time=0.009..0.943 rows=7146 loops=1)		
8	-> Hash (cost=12.50..12.50 rows=24 width=5) (actual time=0.013..0.014 rows=24 loops=1)		
9	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
10	-> Index Only Scan using fd_group_pkey on fd_group fd (cost=0.14..12.50 rows=24 width=5) (actual time=0.003..0.008 rows=24 loops=1)		
11	Heap Fetches: 24		
12	Planning Time: 0.342 ms		
13	Execution Time: 5.368 ms		

After Index:

Create BTree indices to optimize aggregate functions and the join conditions.

```
CREATE INDEX idx_fd_group_fdgrp_cd ON fd_group (fdgrp_cd);
CREATE INDEX idx_food_des_fdgrp_cd ON food_des (fdgrp_cd);
CREATE INDEX idx_food_des_ndb_no ON food_des (ndb_no);
CREATE INDEX idx_food_des_fat_factor ON food_des (fat_factor);
CREATE INDEX idx_food_des_pro_factor ON food_des (pro_factor);
```

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	Aggregate (cost=1085.04..1085.05 rows=1 width=16) (actual time=7.409..7.413 rows=1 loops=1)	
2	-> GroupAggregate (cost=852.79..977.85 rows=7146 width=38) (actual time=5.362..7.180 rows=7146 loops=1)	
3	Group Key: fod.ndb_no	
4	-> Sort (cost=852.79..870.66 rows=7146 width=22) (actual time=5.353..5.567 rows=7146 loops=1)	
5	Sort Key: fod.ndb_no	
6	Sort Method: quicksort Memory: 688kB	
7	-> Hash Join (cost=13.08..395.34 rows=7146 width=22) (actual time=0.019..3.586 rows=7146 loops=1)	
8	Hash Cond: (fod.fmgrp_cd = fd.fmgrp_cd)	
9	-> Index Scan using idx_food_des_ndb_no on food_des fod (cost=0.28..360.47 rows=7146 width=27) (actual time=0.006..1.301 rows=7146 loops=1)	
10	-> Hash (cost=12.50..12.50 rows=24 width=5) (actual time=0.008..0.009 rows=24 loops=1)	
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
12	-> Index Only Scan using idx_fd_group_fmgrp_cd on fd_group fd (cost=0.14..12.50 rows=24 width=5) (actual time=0.002..0.005 rows=24 loops=1)	
13	Heap Fetches: 24	
14	Planning Time: 0.157 ms	
15	Execution Time: 7.447 ms	

Query 24

Before Index:

Data Output Explain Messages Notifications

	QUERY PLAN	
	text	
1	Seq Scan on src_cd (cost=10000000000.00..10000000001.10 rows=10 width=44) (actual time=117.081..117.089 rows=10 loops=1)	
2	Planning Time: 0.084 ms	
3	JIT:	
4	Functions: 2	
5	Options: Inlining true, Optimization true, Expressions true, Deforming true	
6	Timing: Generation 0.336 ms, Inlining 43.275 ms, Optimization 34.264 ms, Emission 38.627 ms, Total 116.502 ms	
7	Execution Time: 161.138 ms	

After Index:

Create a BTree index on srcc_cd to optimize table search.

CREATE INDEX idx_srccd_desc ON src_cd (srccd_desc);

Data Output		Explain	Messages	Notifications
	QUERY PLAN			
	text			
	1	Index Only Scan using idx_src_desc on src_cd (cost=0.14..12.29 rows=10 width=44) (actual time=0.011..0.013 rows=10 loops=1)		
	2	Heap Fetches: 10		
	3	Planning Time: 0.107 ms		
4	Execution Time: 0.021 ms			